当代人工智能实验二：搜索算法

内容：A*（迷宫寻路 / 加权地形）· Minimax 与 Alpha-Beta（博弈树）· 综合题（竞争环境下的宝藏分配）。代码仓库 search-Project2，一键复现：python run_tests.py / run_experiments.py。

1 算法设计与实验设置

A*（astar.py，四道题共用）

状态为网格坐标 (row, col)，通行规则、进入代价、启发函数通过回调注入。g 为起点到当前节点的累计实际代价，h 为到终点的下界估计，f=g+h；最小堆按 f
弹出，堆内附自增计数器做平局裁决（先入先出），保证扩展顺序可复现；关闭表记录各状态最优
g，过期副本弹出时跳过。搜索统计扩展数、生成数、OPEN 峰值。当 h 永不高估真实剩余代价（可采纳）时，A*
首次弹出终点即最优。

Minimax 与 Alpha-Beta（q3_alphabeta.py）

Minimax：叶子回传评估值，MAX 层取最大、MIN 层取最小。Alpha-Beta：递归中维护 [alpha, beta] 区间——MAX 侧 value>=beta 或 MIN 侧 value<=alpha
时，剩余兄弟子树不影响任何祖先决策，直接剪掉。剪枝只减少访问量、不改变返回值；报告用“两算法输出一致”验证正确性，用“求值叶子数/访问节点数”量化剪枝收益。

综合题状态设计（q4_treasure_game.py）

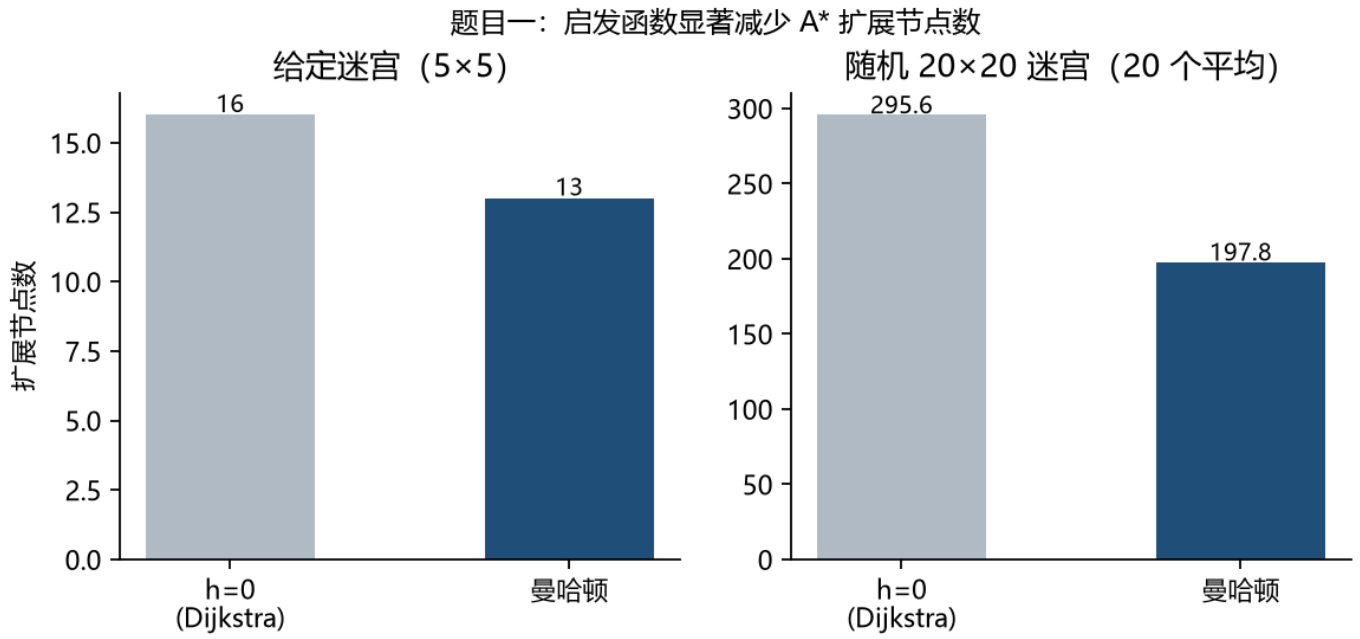
GameState = (A位置, B位置, 剩余宝藏, Score_A, Score_B,
当前玩家)。每回合当前玩家沿最小代价路径移动到所选宝藏，Gain = V_i - d(p, T_i) 入账；到达深度上限或宝藏取完时以 U = Score_A - Score_B 评估。玩家每回合必停在六个点位之一，故先用 A* 预计算 6×6
距离矩阵，博弈搜索中转移代价全部 O(1) 查表。

2 题目一：迷宫寻路

给定 5×5 迷宫 (0,0)→(3,4)，每步代价 1，h 取曼哈顿距离。最优路径长 7
步、恰等于曼哈顿距离，因此最优路径上每个节点 f 恒为 7，A* 几乎直线推进到终点；偏航节点（如 (1,0)，f=9）被压在堆底。程序输出
[(0,0),(0,1),(0,2),(1,2),(1,3),(1,4),(2,4),(3,4)]，与题目预期一致；终点被围死的用例返回空列表。

启发函数	扩展节点数	生成节点数	OPEN 峰值	路径长度
zero	16	18	3	8
manhattan	13	16	3	8

放大到 20 个 20×20 随机迷宫（障碍率 0.25，固定种子）：h=0 平均扩展 295.6 个节点，曼哈顿 197.8 个，减少约 33%。h=0 退化为
Dijkstra，无方向性地层层外扩；曼哈顿把搜索压成朝向终点的椭圆。两种启发找到的路径长度相同——可采纳性保证最优性，差异只在效率。

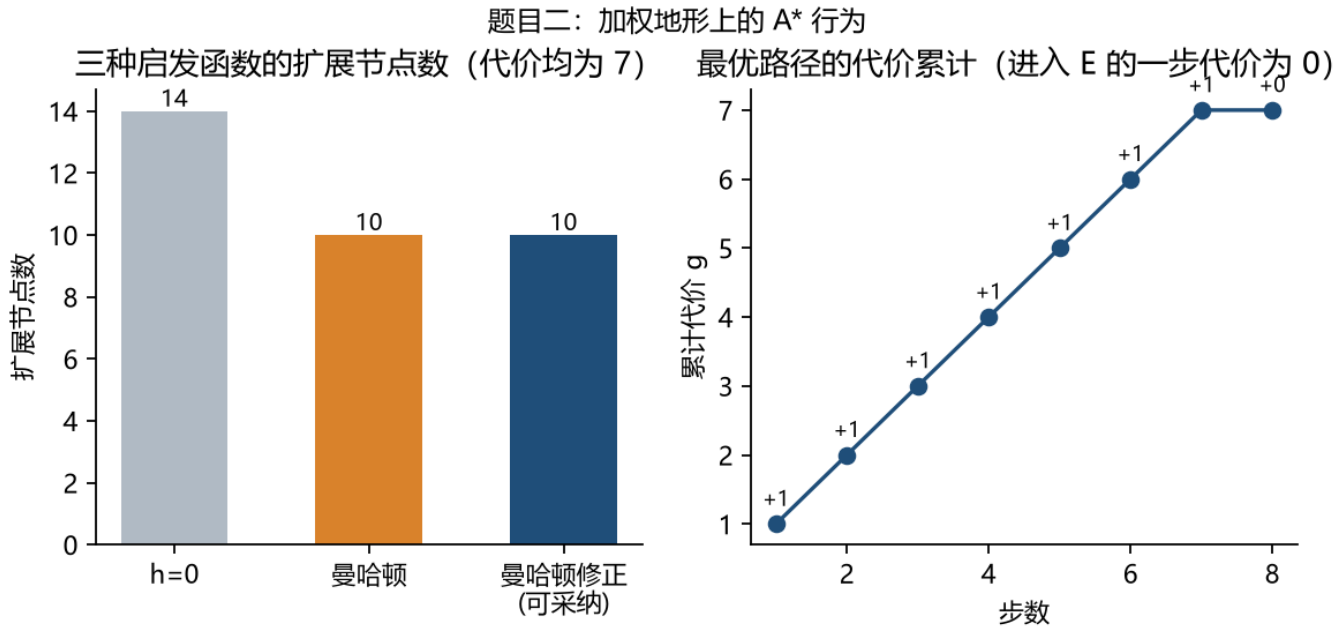


3 题目二：加权地形

进入代价等于地形权值，进入 E 按题目规定计 0。A*
找到的路径与题目预期完全一致：(0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(4,2)→(4,3)→(4,4)，总代价 7。路径绕开 8、9 高代价格：加权地形上“最短”不等于“最便宜”，g 的真实累计与 h 的下界估计缺一不可。

进入格	地形	进入代价	累计 g
(1, 0)	1	1	1
(2, 0)	1	1	2
(3, 0)	1	1	3
(3, 1)	1	1	4
(3, 2)	1	1	5
(4, 2)	1	1	6
(4, 3)	1	1	7
(4, 4)	E	0	7

可采纳性讨论：地图最小权值为 1，看似曼哈顿可采纳；但“进入 E 免费”使朴素曼哈顿最多高估 1——反例 [[S,E]]：h(S)=1 而真实剩余代价为 0。修正为 h'(n)=max(manhattan-1, 0)×min_cost，任意 L 步路径代价 ≥ (L-1)×1+0，故 h' 严格可采纳。给定地图上三种启发结果一致（均为 7，扩展数 14/10/10），高估未改变扩展顺序，但只有修正版提供最优性的理论保证。

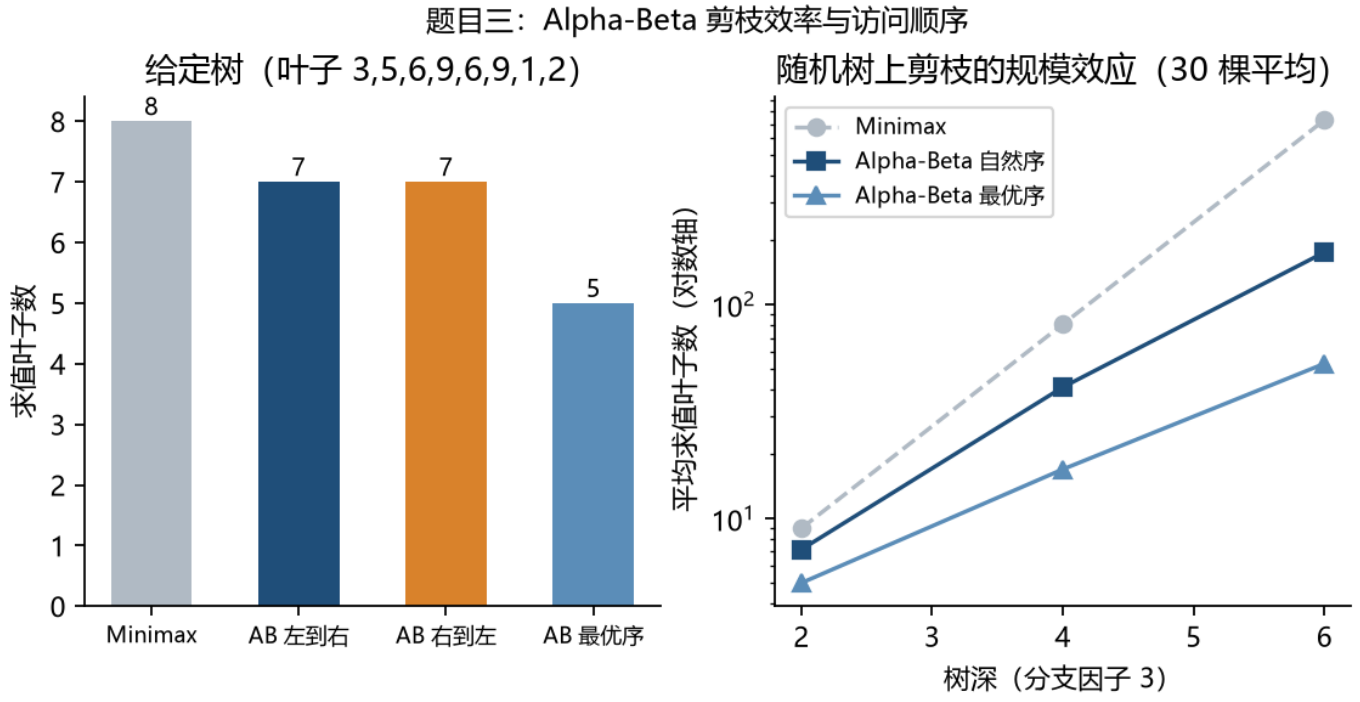


4 题目三：Alpha-Beta 剪枝

给定树（叶子从左到右 3,5,6,9,6,9,1,2）：三个 MAX 节点回传 5、9、2，两个 MIN 节点回传 min(5,9)=5、min(9,2)=2，根 MAX 取 max(5,2)=5，符合预期。剪枝过程：左 MIN 子树定值 5 后根 alpha=5；进入右 MIN 子树，其第二个 MAX 孩子先看到叶子 1，右 MIN ≤1 ≤ alpha，根不会选右侧，叶子 2 被剪掉。Alpha-Beta 返回 5，与 Minimax 相同，少求值 1 个叶子。

方法	求值叶子数	访问节点总数	剪枝次数
Minimax（无剪枝）	8	15	0
Alpha-Beta 从左到右	7	14	1
Alpha-Beta 从右到左	7	14	1
Alpha-Beta 最优顺序	5	11	2

放大到随机博弈树（分支因子 3、叶子随机 1–9、30 棵平均）：深度 2/4/6 时 Minimax 求值 9/81/729 个叶子，Alpha-Beta 自然序 7.2/41.1/175.7，最优序 5/17/53。深度 6 时自然序只访问约 24% 叶子、最优序仅约 7%——若总先访问好棋，叶子访问量约 O(b^(d/2))，相当于把有效搜索深度翻倍。访问顺序是剪枝收益的开关。



5 题目四：竞争环境下的宝藏分配（综合题）

进入代价：数字格取其值，A/B/T 格为 1；h=曼哈顿（每步至少花 1，可采纳）。15 对点位各跑一次 A*（矩阵对称只算一半），共扩展 169 个节点。例如 A→T1 因 (0,3)=X 绕行第 2 行全 1 通道，代价 10；A—B、T1—T4 的 12 恰等于曼哈顿距离，说明绕行未产生额外代价。

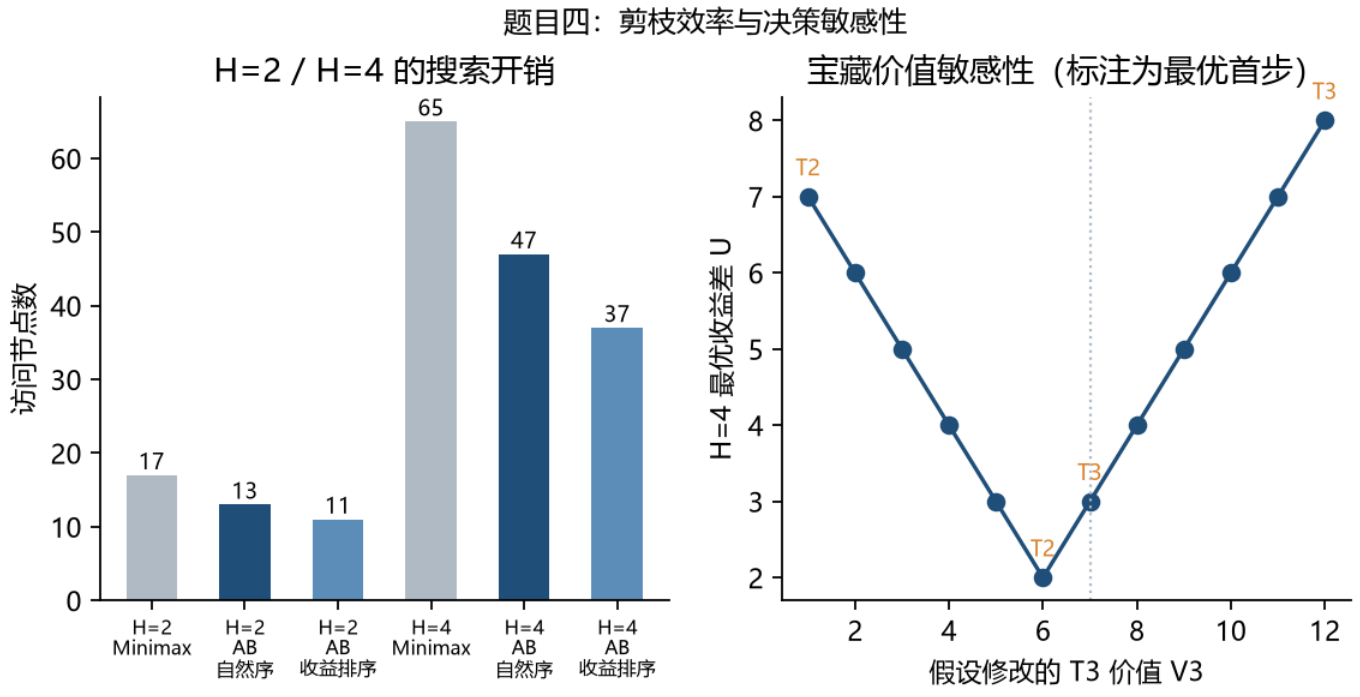
	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

状态 GameState 记录双方位置、剩余宝藏、双方累计收益与当前玩家（见第 1 节）。H=2 与 H=4 下，Minimax 与 Alpha-Beta 输出完全一致，并与题目预期一致：

H	方法	最优首步	最优 U	访问节点	评估次数	剪枝数
2	Minimax	T2	-1	17	12	0
2	Alpha-Beta 自然序	T2	-1	13	8	4
2	Alpha-Beta 收益排序	T2	-1	11	6	6
4	Minimax	T3	3	65	24	0
4	Alpha-Beta 自然序	T3	3	47	16	6
4	Alpha-Beta 收益排序	T3	3	37	12	8

决策差异：H=2 只看两步，A 首步拿即时收益最高的 T2（6-4=+2），随后 B 必拿 家门口的 T3（7-4=+3），U=-1；H=4 看到全局——T3 卡在 B 家门口，不只是收益，更是必须抢走的战略点，A→T3 后终局 U=+3（逐首步：T1→-5, T2→+1, T3→+3, T4→-7）。深度扫描 H=1,2,3 最优首步均为 T2，只有完整对局让 T3 的战略价值显形：评估函数的短视无法被更聪明的搜索弥补，深度本身就是信息。

剪枝效果：收益排序（当前玩家按即时 Gain 从大到小生成走法）把 H=4 访问节点从 47 降到 37（-21%），较 Minimax 的 65 累计减少 43%；输出不变，是纯效率优化。



6 自设对照实验与拓展

决策边界：把 V3 从 1 扫到 12（H=4, Minimax），V3≤6 时最优首步为 T2、V3≥7 时为 T3——题目给定值 V3=7 恰好落在决策边界上，差 1 分答案就换成 T2，说明题目参数经过设计（上图右）。

距离缓存：预计算 15 对 A* 距离共约 0.33 ms；若在博弈搜索中每次转移重新调用 A*，H=4 Minimax 的 64 次转移约需 1.42 ms，且随深度成倍增长。预计算把转移代价降为 O(1) 查表——玩家位置只可能落在 6 个点位上，距离查询集合是封闭的，这是状态空间结构化带来的红利。

健壮性用例（自设，均纳入 run_tests.py 回归）：终点被障碍围死时题目一返回 []、题目二返回 -1；[[S,E]] 反例验证朴素曼哈顿高估而修正启发 h'=0 恰等于真实代价；题目三在左到右/右到左/最优序下 Alpha-Beta 返回值均与 Minimax 一致。

7 结论

1）A* 的效率来自启发的方向性：可采纳的 h 保证最优，h 越紧扩展越少（给定迷宫 16→13，随机迷宫 -33%）。

2）加权地形把“最短”变成“最便宜”；终点免费的规则会使朴素曼哈顿失去可采纳性，需要 -1 修正，给定地图上修正前后结果一致但只有修正版提供理论保证。

3）Alpha-Beta 与 Minimax 输出恒等，收益全在访问量：给定树 8→7 叶子，随机深 6 树 -76%，最优访问序 -93%；先访问好棋是剪枝的杠杆。

4）综合题展示两种搜索的分工：A* 把地图压缩成 6×6 代价矩阵，博弈搜索在矩阵上做决策。H=2 的短视最优（T2, -1）不同于 H=4 的全局最优（T3, +3）——对手的存在让“近在咫尺的高收益”成为诱饵，而 V3=7 恰在决策边界上。

可复现性：python run_tests.py（8 项断言全部 PASS）；python run_experiments.py 重新生成 results/ 下全部数据与图；随机实验固定种子（迷宫 42、博弈树 7），多次运行数值一致。算法实现仅依赖 Python 标准库。